

UP LT GRADE • COMPUTER SCIENCE MAINS

# Digital Logic and Circuits & Discrete Mathematical Structures

*Complete Descriptive-Exam Notes — Simple, Clear, Exam-Ready*

---

**Unit 1 — Digital Logic and Circuits:** Number Systems • Boolean Algebra & Logic Gates • Simplification of Boolean Functions • Combinational Circuits • Sequential Circuits • Memory Circuits

**Unit 2 — Discrete Mathematical Structures:** Sets, Relations & Functions • Mathematical Logic • Boolean Algebra • Combinatorics & Recurrence Relations • Graph Theory

Prepared for descriptive-exam revision • 45-day study plan

# Table of Contents

---

## Unit 1: Digital Logic and Circuits

|                                        |         |
|----------------------------------------|---------|
| 1. Number Systems                      | Page 3  |
| 2. Boolean Algebra and Logic Gates     | Page 5  |
| 3. Simplification of Boolean Functions | Page 7  |
| 4. Combinational Circuits              | Page 9  |
| 5. Sequential Circuits                 | Page 11 |
| 6. Memory Circuits                     | Page 13 |

## Unit 2: Discrete Mathematical Structures

|                                          |         |
|------------------------------------------|---------|
| 7. Sets, Relations & Functions           | Page 15 |
| 8. Mathematical Logic                    | Page 17 |
| 9. Boolean Algebra (Algebraic Structure) | Page 19 |
| 10. Combinatorics & Recurrence Relations | Page 21 |
| 11. Graph Theory                         | Page 23 |

# 1. Number Systems

## In Simple Words

Computers only understand two states (on/off), so they represent everything in **binary**. Number systems are just different "bases" for writing the same quantity — decimal (what we use daily), binary, octal, and hexadecimal (what computers and programmers use).

## Key Number Systems

| System      | Base | Digits Used | Example |
|-------------|------|-------------|---------|
| Binary      | 2    | 0, 1        | 1010    |
| Octal       | 8    | 0–7         | 17      |
| Decimal     | 10   | 0–9         | 10      |
| Hexadecimal | 16   | 0–9, A–F    | A       |

## Conversion Rules

- **Decimal → Any base:** Repeatedly divide by the base, collect remainders bottom-to-top.
- **Any base → Decimal:** Multiply each digit by  $(\text{base})^{\text{position}}$  and sum.
- **Binary ↔ Octal:** Group binary digits in sets of 3 (from right).
- **Binary ↔ Hex:** Group binary digits in sets of 4 (from right).

### Solved Example

Convert  $(25)_{10}$  to binary:

$25 \div 2 = 12 \text{ rem } 1 \rightarrow 12 \div 2 = 6 \text{ rem } 0 \rightarrow 6 \div 2 = 3 \text{ rem } 0 \rightarrow 3 \div 2 = 1 \text{ rem } 1 \rightarrow 1 \div 2 = 0 \text{ rem } 1$

Reading remainders bottom-to-top:  $(11001)_2$

## Representing Negative Numbers

- **Sign-Magnitude:** First bit = sign (0 positive, 1 negative), rest = magnitude. Has two representations of zero.
- **1's Complement:** Flip every bit of the positive number. Also has two zeros (+0 and -0).
- **2's Complement:** Take 1's complement and add 1. **Used in virtually all real computers** — has only one representation of zero and makes subtraction = addition of the negative.

### Must-Remember Point

2's Complement of a binary number = (1's Complement) + 1. This is why subtraction  $A - B$  is done as  $A + (2\text{'s complement of } B)$  inside the ALU — no separate subtractor circuit is needed!

## BCD (Binary Coded Decimal)

- Each decimal digit (0–9) is individually converted to its 4-bit binary form.
- Example:  $(25)_{10}$  in BCD = 0010 0101 (NOT the same as pure binary 11001).
- Used in digital clocks, calculators — easier to display decimal digits directly.

**Exam Tip**

Practice at least 2–3 conversions in each direction (decimal  $\leftrightarrow$  binary  $\leftrightarrow$  octal  $\leftrightarrow$  hex) and one 2's complement subtraction problem — these are almost guaranteed numerical questions.

## 2. Boolean Algebra and Logic Gates

### In Simple Words

Boolean algebra is mathematics with only two values — **True (1)** and **False (0)**. Logic gates are the physical electronic circuits that carry out Boolean operations like AND, OR, and NOT on electrical signals.

### Basic Logic Gates

| Gate | Symbol/Expression   | Rule                                     |
|------|---------------------|------------------------------------------|
| AND  | $Y = A \cdot B$     | Output is 1 only if ALL inputs are 1     |
| OR   | $Y = A + B$         | Output is 1 if ANY input is 1            |
| NOT  | $Y = \bar{A}$       | Inverts the input (1→0, 0→1)             |
| NAND | $Y = (A \cdot B)'$  | Opposite of AND; a <b>universal gate</b> |
| NOR  | $Y = (A + B)'$      | Opposite of OR; a <b>universal gate</b>  |
| XOR  | $Y = A \oplus B$    | Output is 1 only if inputs are DIFFERENT |
| XNOR | $Y = (A \oplus B)'$ | Output is 1 only if inputs are SAME      |

### Why NAND and NOR Are "Universal"

- Any Boolean function (AND, OR, NOT, or any combination) can be built using **only NAND gates**, or **only NOR gates**.
- This is extremely useful in IC manufacturing — factories can mass-produce just one gate type and build everything from it.

### Important Boolean Laws

| Law            | Rule                                      |
|----------------|-------------------------------------------|
| Identity       | $A + 0 = A$ $A \cdot 1 = A$               |
| Null           | $A + 1 = 1$ $A \cdot 0 = 0$               |
| Idempotent     | $A + A = A$ $A \cdot A = A$               |
| Complement     | $A + \bar{A} = 1$ $A \cdot \bar{A} = 0$   |
| Commutative    | $A + B = B + A$ $A \cdot B = B \cdot A$   |
| Distributive   | $A \cdot (B + C) = A \cdot B + A \cdot C$ |
| De Morgan's I  | $(A + B)' = \bar{A} \cdot \bar{B}$        |
| De Morgan's II | $(A \cdot B)' = \bar{A} + \bar{B}$        |

#### De Morgan's Theorem — Most Asked Law

"The complement of a sum equals the product of complements, and vice versa." This single law is the basis for converting AND/OR circuits into NAND/NOR-only circuits — expect at least one direct question on this.

**Exam Tip**

Draw the truth table AND the gate symbol together whenever asked to "explain" a gate — descriptive answers scoring full marks always include both.

## 3. Simplification of Boolean Functions

### In Simple Words

A Boolean expression taken directly from a truth table is often long and uses more gates than necessary. Simplification reduces it to the shortest equivalent expression — fewer gates means a cheaper, faster, and less power-hungry circuit.

### Methods of Simplification

- **Algebraic Method:** Apply Boolean laws (from Topic 2) step-by-step to reduce the expression. Works but easy to miss a shorter path.
- **Karnaugh Map (K-Map):** A grid where adjacent cells differ by only 1 bit. You group adjacent 1s into the largest possible groups of size 1, 2, 4, 8... to read off the simplest expression directly.

### K-Map Grouping Rules

- Groups must have a size that is a power of 2 (1, 2, 4, 8...).
- Groups can be horizontal, vertical, or wrap around the edges of the map (K-Maps are cylindrical/toroidal).
- Always make groups **as large as possible** — larger groups eliminate more variables.
- Every 1 must be covered by at least one group; overlapping groups are allowed.

### Solved Example

For a 2-variable function  $F(A,B) = \Sigma(1,2,3)$  [minterms where output = 1]:

Truth table: 00→0, 01→1, 10→1, 11→1.

Grouping (01,11) gives B; grouping (10,11) gives A. Since these two groups cover all 1s:

$F = A + B$  (much simpler than the original 3-term sum-of-products expression).

### Related Terms

- **Minterm:** A product term where every variable appears once (true or complemented) — represents exactly one row where output = 1.
- **Maxterm:** A sum term representing exactly one row where output = 0.
- **SOP (Sum of Products):** OR of AND terms — built from minterms.
- **POS (Product of Sums):** AND of OR terms — built from maxterms.
- **Don't Care conditions (X):** Input combinations that never occur in practice — can be treated as either 0 or 1, whichever helps simplify the expression more.

### Exam Tip

K-Maps up to 4 variables are the most commonly asked. Always draw the grid neatly, circle your groups clearly, and write the final simplified expression below — partial credit is given even for correct grouping with a small error.

## 4. Combinational Circuits

### In Simple Words

A combinational circuit's output depends **only on the current inputs** — it has no memory of past inputs. Change the input, and the output changes immediately (after gate delay). It's built purely from logic gates with no feedback loop.

### Key Combinational Circuits

- **Half Adder:** Adds two single bits; outputs Sum (XOR) and Carry (AND). Cannot handle a carry-in from a previous stage.
- **Full Adder:** Adds two bits PLUS a carry-in from the previous stage; outputs Sum and Carry-out. Built by combining two half adders and an OR gate.
- **Multiplexer (MUX):** Selects ONE of many input lines and routes it to a single output, based on select lines (a "data selector"). A 4-to-1 MUX needs 2 select lines ( $2^2=4$ ).
- **Demultiplexer (DEMUX):** The reverse of MUX — takes ONE input and routes it to one of many outputs based on select lines.
- **Decoder:** Converts  $n$  input lines into up to  $2^n$  unique output lines (only one output is active at a time).
- **Encoder:** The reverse of a decoder — converts one of  $2^n$  active input lines into an  $n$ -bit binary code.

### Half Adder vs Full Adder

Half Adder:  $\text{Sum} = A \oplus B$ ,  $\text{Carry} = A \cdot B$  (2 inputs only)

Full Adder:  $\text{Sum} = A \oplus B \oplus \text{Cin}$ ,  $\text{Carry-out} = AB + \text{Cin}(A \oplus B)$  (3 inputs: A, B, Cin)

### Why This Matters

Chaining multiple Full Adders together (each one's Carry-out feeding the next one's Carry-in) creates a **Ripple Carry Adder** — the basic circuit an ALU uses to add multi-bit binary numbers.

### Exam Tip

Draw the Full Adder's truth table AND circuit diagram together — a very frequently asked full-length descriptive question. Also memorize the MUX select-line formula:  $n$  select lines give  $2^n$  inputs.



# 5. Sequential Circuits

**In Simple Words**

Unlike combinational circuits, a sequential circuit's output depends on **both the current input AND the past state** — it has memory. This memory is achieved using feedback and a basic storage element called a flip-flop, usually synchronized by a clock signal.

**Types of Flip-Flops**

| Flip-Flop      | Behavior                                                                                      |
|----------------|-----------------------------------------------------------------------------------------------|
| SR (Set-Reset) | S=1 sets output to 1; R=1 resets to 0; S=R=1 is an invalid/forbidden state.                   |
| D (Data/Delay) | Output simply follows the D input on each clock pulse — used to eliminate SR's invalid state. |
| JK             | Improved SR — when J=K=1, output simply TOGGLES instead of being invalid.                     |
| T (Toggle)     | Output toggles (flips) every clock pulse when T=1; holds when T=0.                            |

**Circuits Built from Flip-Flops**

- **Registers:** A group of flip-flops used to store multiple bits together (e.g., an 8-bit register stores one byte).
- **Counters:** Circuits that go through a predefined sequence of states on each clock pulse — e.g., a 4-bit binary counter counts 0000 to 1111 and repeats.
  - **Asynchronous (Ripple) Counter:** Each flip-flop's clock is triggered by the previous flip-flop's output — simple but slower (delay accumulates).
  - **Synchronous Counter:** All flip-flops share the same clock signal — faster, more complex wiring.
- **Finite State Machines (FSM):** Model of a system that moves between a fixed number of states based on inputs — Moore machine (output depends only on state) vs Mealy machine (output depends on state AND input).

**Exam Tip**

Always contrast Combinational vs Sequential circuits directly if asked to "differentiate" — the presence/absence of memory and clock is the core distinguishing answer examiners look for.

## 6. Memory Circuits

**In Simple Words**

Memory circuits are the actual hardware building blocks used to store bits. The smallest possible memory circuit is a single flip-flop (1 bit); combining millions of them (or capacitor-based cells) creates the RAM and ROM chips in a real computer.

**RAM (Random Access Memory) — Volatile**

| Type           | How It Stores Data                                   | Speed / Use                                        |
|----------------|------------------------------------------------------|----------------------------------------------------|
| SRAM (Static)  | Uses flip-flops (transistor pairs)                   | Very fast, expensive — used as CPU cache           |
| DRAM (Dynamic) | Uses a capacitor per bit — needs periodic refreshing | Slower, cheaper — used as main memory (RAM sticks) |

**ROM (Read-Only Memory) — Non-Volatile**

- **PROM:** Programmable once by the user (using a special device); cannot be changed after.
- **EPROM:** Erasable using ultraviolet light, then reprogrammable.
- **EEPROM:** Erasable electrically (byte-by-byte) — the basis of modern Flash memory.

**Key Distinguishing Fact**

Volatile memory (RAM) loses all data when power is switched off. Non-volatile memory (ROM/Flash) retains data even without power — this is why BIOS/firmware is stored in ROM, not RAM.

**Exam Tip**

A table comparing SRAM vs DRAM and RAM vs ROM (as shown above) is a high-scoring answer format — examiners like clear tabular comparisons for "differentiate between" questions.

## 7. Sets, Relations & Functions

### In Simple Words

A **set** is just a well-defined collection of distinct objects. A **relation** connects elements of one set to another. A **function** is a special, "well-behaved" relation where every input has exactly one output.

### Set Operations

| Operation         | Meaning                                     | Notation          |
|-------------------|---------------------------------------------|-------------------|
| Union             | Elements in A OR B                          | $A \cup B$        |
| Intersection      | Elements in A AND B                         | $A \cap B$        |
| Difference        | Elements in A but not B                     | $A - B$           |
| Complement        | Elements not in A (within universal set)    | $\bar{A}$ or $A'$ |
| Cartesian Product | All ordered pairs (a,b), $a \in A, b \in B$ | $A \times B$      |

### Types of Relations

- **Reflexive:** Every element is related to itself:  $(a,a) \in R$  for all  $a$ .
- **Symmetric:** If  $(a,b) \in R$  then  $(b,a) \in R$ .
- **Transitive:** If  $(a,b) \in R$  and  $(b,c) \in R$ , then  $(a,c) \in R$ .
- **Equivalence Relation:** A relation that is reflexive, symmetric, AND transitive — it partitions a set into disjoint "equivalence classes."

### Types of Functions

- **One-One (Injective):** Different inputs always give different outputs.
- **Onto (Surjective):** Every element of the codomain is hit by at least one input.
- **Bijective:** Both one-one AND onto — a perfect pairing; only bijective functions have a proper inverse function.

#### Solved Example

Let  $A = \{1,2\}$ ,  $B = \{3,4\}$ . Then  $A \times B = \{(1,3), (1,4), (2,3), (2,4)\}$ . A relation  $R = \{(1,3), (2,4)\}$  from A to B is a function since each input (1 and 2) maps to exactly one output.

#### Exam Tip

When asked to prove a relation is an equivalence relation, always check and state all THREE properties explicitly (reflexive, symmetric, transitive) — partial checking loses marks.

## 8. Mathematical Logic

### In Simple Words

Mathematical logic is the formal, symbolic study of reasoning. A **proposition** is any statement that is definitely either True or False (never both, never neither). Logic lets us combine propositions and check whether an argument is valid.

### Logical Connectives

| Connective    | Symbol                | Meaning                                         |
|---------------|-----------------------|-------------------------------------------------|
| Negation      | $\neg p$              | NOT p                                           |
| Conjunction   | $p \wedge q$          | p AND q (true only if both true)                |
| Disjunction   | $p \vee q$            | p OR q (true if at least one true)              |
| Implication   | $p \rightarrow q$     | "if p then q" (false only when p true, q false) |
| Biconditional | $p \leftrightarrow q$ | "p if and only if q" (true when both match)     |

### Classifying Compound Statements

- **Tautology:** A statement that is TRUE for every possible input combination (e.g.,  $p \vee \neg p$ ).
- **Contradiction:** A statement that is FALSE for every possible input combination (e.g.,  $p \wedge \neg p$ ).
- **Contingency:** A statement that is sometimes true, sometimes false, depending on inputs.

### Quantifiers (Predicate Logic)

- **Universal Quantifier ( $\forall$ ):** "for all" — e.g.,  $\forall x P(x)$  means  $P(x)$  is true for every  $x$ .
- **Existential Quantifier ( $\exists$ ):** "there exists" — e.g.,  $\exists x P(x)$  means  $P(x)$  is true for at least one  $x$ .

#### Solved Example

Check if  $p \rightarrow q$  is equivalent to  $\neg p \vee q$  using a truth table: both expressions give the exact same True/False pattern for every combination of  $p$  and  $q$  — confirming the well-known logical equivalence used to rewrite implications.

#### Exam Tip

Constructing a full truth table (showing all rows) is usually mandatory in descriptive answers when asked to prove a logical equivalence or classify a statement as tautology/contradiction.

## 9. Boolean Algebra (as an Algebraic Structure)

### In Simple Words

This is the same Boolean algebra from Unit 1 (Topic 2), but here it's studied as an **abstract mathematical structure** — a set with two operations satisfying specific axioms — which is also the foundation of **Lattice Theory** in discrete mathematics.

### Formal Definition

A Boolean Algebra is a set  $B$  with two binary operations  $(+, \cdot)$ , a unary operation (complement), and two distinct elements  $0$  and  $1$ , satisfying:

- **Closure:** Results of  $+$  and  $\cdot$  stay within  $B$ .
- **Commutativity & Associativity** of both operations.
- **Distributive Laws** of  $+$  over  $\cdot$  and  $\cdot$  over  $+$ .
- **Identity elements:**  $0$  for  $+$ ,  $1$  for  $\cdot$ .
- **Complement:** every element  $a$  has a complement  $\bar{a}$  such that  $a + \bar{a} = 1$  and  $a \cdot \bar{a} = 0$ .

### Connection to Lattices

- A **lattice** is a partially ordered set where every pair of elements has a unique Least Upper Bound (join,  $\vee$ ) and Greatest Lower Bound (meet,  $\wedge$ ).
- Every Boolean Algebra is a special kind of lattice — specifically, a **complemented, distributive lattice**.
- This connects the "gates and circuits" version of Boolean algebra to the more abstract, proof-based version used in discrete mathematics.

### Exam Tip

If a question asks to "prove  $B$  is a Boolean Algebra," verify each axiom (closure, identity, complement, distributive) one at a time in a numbered list — this structured approach scores full marks even under time pressure.

# 10. Combinatorics & Recurrence Relations

**In Simple Words**

Combinatorics is the mathematics of **counting** without listing every possibility one by one. Recurrence relations describe a sequence where each term is defined using earlier terms — like a mathematical "loop."

**Permutations vs Combinations**

| Concept               | Order Matters? | Formula            |
|-----------------------|----------------|--------------------|
| Permutation ( $nPr$ ) | Yes            | $n! / (n-r)!$      |
| Combination ( $nCr$ ) | No             | $n! / [r! (n-r)!]$ |

**Solved Example**

Ways to choose 3 students from 5 for a team (order doesn't matter):  $5C3 = 5!/(3!2!) = 10$  ways.  
Ways to arrange 3 of those 5 students in ranked positions (order matters):  $5P3 = 5!/2! = 60$  ways.

**Pigeonhole Principle**

If you distribute more than  $n$  items into  $n$  containers, at least one container must hold 2 or more items. Sounds obvious, but it's used to prove surprisingly deep results (e.g., in any group of 13 people, at least two share a birth month).

**Recurrence Relations**

- A recurrence defines each term using previous terms, e.g. the **Fibonacci sequence**:  $F(n) = F(n-1) + F(n-2)$ , with  $F(0)=0$ ,  $F(1)=1$ .
- Solving method (Linear, constant coefficients)**: Form the "characteristic equation," find its roots, then write the general solution using those roots, and finally apply initial conditions to find exact constants.

**Characteristic Equation Example**

For recurrence  $a_n = 5a_{n-1} - 6a_{n-2}$ , the characteristic equation is  $x^2 - 5x + 6 = 0$ , giving roots  $x = 2, 3$ . General solution:  $a_n = A(2^n) + B(3^n)$ , where  $A$  and  $B$  are found from initial conditions.

**Exam Tip**

Always state the formula first, substitute values clearly step-by-step, and box/underline your final numeric answer — combinatorics questions are marked heavily on correct method, not just the final number.

# 11. Graph Theory

### In Simple Words

A **graph** is simply a set of points (called **vertices**) connected by lines (called **edges**). Graphs are used to model networks — social networks, road maps, computer networks, scheduling problems, and much more.

### Types of Graphs

- **Directed vs Undirected:** Directed graph edges have a direction (arrows); undirected edges don't.
- **Weighted vs Unweighted:** Weighted graphs assign a cost/value to each edge (e.g., distance).
- **Simple Graph:** No loops (edge from a vertex to itself) and no multiple edges between the same pair.
- **Multigraph:** Allows multiple edges between the same pair of vertices.

### Key Terminology

- **Degree of a Vertex:** Number of edges connected to it. In a directed graph: in-degree (incoming) + out-degree (outgoing).
- **Path:** A sequence of edges connecting a sequence of distinct vertices.
- **Cycle:** A path that starts and ends at the same vertex.
- **Connected Graph:** There is a path between every pair of vertices.
- **Tree:** A connected graph with NO cycles — has exactly  $(n-1)$  edges for  $n$  vertices.

### Special Paths

| Type                     | Definition                                                  |
|--------------------------|-------------------------------------------------------------|
| Eulerian Path/Circuit    | Visits every EDGE exactly once (circuit returns to start)   |
| Hamiltonian Path/Circuit | Visits every VERTEX exactly once (circuit returns to start) |

### Important Algorithms

- **Dijkstra's Algorithm:** Finds the shortest path from one source vertex to all others in a weighted graph (no negative weights).
- **Kruskal's / Prim's Algorithm:** Find a Minimum Spanning Tree (MST) — the cheapest set of edges connecting all vertices with no cycles.
- **Graph Coloring:** Assigning colors to vertices so that no two adjacent vertices share a color — used in scheduling and map coloring problems.

### Handshaking Theorem

The sum of degrees of all vertices in a graph =  $2 \times$  (number of edges). This is because every edge contributes exactly 2 to the total degree count (one for each endpoint).

### Exam Tip

Draw a small labeled graph example whenever explaining a term (degree, cycle, tree, Eulerian path) — visual examples are what separates a 3-mark answer from a 5-mark answer in descriptive papers.

— End of Notes —

Digital Logic and Circuits & Discrete Mathematical Structures | UP LT Grade CS Mains Preparation